THE VELVET FOUNDRY

WHERE YOUR AWS BILL GOES



THE VELVET FOUNDRY

Free sample

WHERE YOUR AWS BILL GOES

The five shapes of cloud cost, the decisions that set them, and a monthly review that takes an hour

This sample contains the contents, the opening of the book and one full chapter ("Chapter 3. The five things you actually pay for"), exactly as they appear in the full book. The full book has 68 pages. Page numbers in the contents refer to the full book.

Get the full book:

https://thevelvetfoundry.gumroad.com/l/where-your-aws-bill-goes

Independent guide. Not affiliated with or endorsed by Amazon Web Services. Contains no prices, rates or figures, by design.

Contents

How to use this book 5

Chapter 1 What this book does not do 7

Chapter 2 Why cloud bills surprise people 8

Chapter 3 The five things you actually pay for 10

Chapter 4 Reading the bill as a structure 13

Chapter 5 Tagging, and why attribution beats optimisation 17

Chapter 6 Accounts, organisations and consolidated billing 20

Chapter 7 Data transfer, the cost nobody models 23

Chapter 8 The appliance that charges per gigabyte 26

Chapter 9 Idle, forgotten and orphaned 28

Chapter 10 Storage, retention and the cost of keeping things 31

Chapter 11 Provisioned capacity and commitments 34

Chapter 12 Right-sizing without guessing 36

Chapter 13 Managed databases, where the cost actually sits 40

Chapter 14 Serverless and always-on have different cost shapes 43

Chapter 15 Environments, and how non-production escapes 45

Chapter 16 Budgets, alarms and anomalies 47

Chapter 17 The monthly cost review, in one hour 50

Chapter 18 A month, worked end to end 53

Chapter 19 Costing a design before you build it 56

Chapter 20 When to stop optimising 59

Conclusion: the four habits that matter 61

Recommended next steps and further resources 63

About the publisher 67

Copyright 68

How to use this book

Somebody forwarded you the AWS invoice and asked what happened. You opened the console, found a number, and could not explain it. Not because you are careless — because almost nothing about a cloud bill is arranged to answer that question.

This book is about the arrangement. It explains what AWS actually charges for, which architectural decisions quietly set those charges, where the cost hides when you go looking for it, and the monthly routine that turns a surprising bill into a boring one.

By the end you will have a method for tracing a line on your bill back to the decision that most likely caused it, a routine for finding the few things driving most of your spend that is designed to fit in about an hour, and a way to tell the difference between a cost that is a problem and a cost that is simply the price of running the thing.

Read it in this order

Chapters 1 to 4 are the foundation: what this book does and does not cover, why bills surprise people, the five things you are actually paying for, and how to read the bill as a structure rather than a total. Skipping them makes everything after look like a list of tricks.

Chapters 5 and 6 are about attribution: tags, and accounts. Chapters 7 to 15 are where the money usually is, roughly in order of how often each one is the answer. Data transfer first, because it is the cost almost nobody models.

Chapters 16 to 20 are the practice: budgets and alarms, the monthly review, a worked month, costing a design before you build it, and the judgement call about when to stop.

Work through it with your own Cost Explorer open. Reading it without doing it will feel like it worked and will not stick.

What you need

An AWS account you can see billing data for, and the IAM permission to read Cost Explorer and the billing console — which is itself often the first obstacle, and chapter 4 deals with it. A spreadsheet. Nothing else, and nothing to buy.

If you have just inherited the account

A special case, because it is common and disorienting. You have been handed an AWS account somebody else built, with resources you did not create and no documentation.

Do three things before anything else. Switch on Cost Explorer, if it is not already on. AWS then prepares your cost data for the current month and the previous 13 months: the current month is visible in about 24 hours and the rest takes a few days longer [S1]. Check which Cost Anomaly Detection monitors and alert subscriptions already exist, and make sure an alert on the whole account reaches a person. Detection can take up to a day, so treat it as a safety net rather than a live alarm [S1, S9]. Then read chapter 4 and group a single month by service, which will tell you in about ten minutes what kind of estate you have inherited.

Do not optimise anything in the first week. An account you do not understand is one where the idle resource and the load-bearing one look identical.

Who this is not for

If you are a cloud financial management specialist with a dedicated team and a commitment portfolio to manage, this book is below you and you should read the vendor's own framework instead.

If you want somebody to tell you which instance to buy, this book will frustrate you. It teaches you to answer that yourself, because the answer depends on measurements only you have.

And if you are looking for a list of tricks to apply without understanding them, the ordering here will seem perverse: the most interesting chapters are deliberately last, because the boring ones save more money.

This is for the engineer or the small team who owns an AWS bill they did not entirely design, and who would like to be able to explain it.

One promise and one warning

The promise: cloud cost is not mysterious. It is a small number of pricing shapes applied to a large number of services. Once you can recognise the shapes, a service you have never used before becomes readable.

The warning: this book contains no prices. Not one. The next chapter explains why that is deliberate, and why a book full of prices would be actively worse for you. It does name the AWS services involved, so you can find them on your own bill, and it quotes a few documented limits and time windows. Those are facts about how AWS behaves, not prices.

How the facts in this book were checked

Statements about AWS behaviour were checked against the official AWS documentation on 2026-11-07. Where a statement rests on a documented behaviour, the text carries a source tag such as [S1]. The matching address for each tag is in the last section of the book. AWS changes its services without notice, so open the linked page before you rely on anything here.

CHAPTER 1

What this book does not do

Read this chapter. It is short and it sets the boundaries honestly.

This book contains no prices, no rates and no cost figures. You will not find a per-gigabyte charge, an hourly rate, a free-tier limit or a worked example with real currency in it. That is a deliberate design decision, not an omission. The book does name services (S3, EC2 and EBS, Lambda, CloudWatch and others) so that you can recognise them on your bill, and it states a few documented behaviours such as how many months of history Cost Explorer prepares. Those are behaviours, dated and sourced, not prices.

AWS prices differ by region, change without a book being reprinted, vary by commitment and agreement, and in several services depend on usage tiers that reset monthly. A book that stated them would be wrong for most readers the day it shipped, and wrong in a way that looked authoritative — which is worse than saying nothing, because you would plan against it.

So this book teaches the part that does not expire: **what you are charged for, and which decisions change how much of it you consume.** The current number for your region and account is always one click away in the AWS pricing pages and the AWS Pricing Calculator, both published by AWS and both authoritative in a way this book could never be.

Put plainly: this book tells you *where* the money goes and *why*. AWS tells you *how much*.

What else is out of scope. Negotiated enterprise agreements, private pricing and marketplace contracts are between you and AWS. Tax treatment of cloud spend is a question for your accountant. Other cloud providers are not covered; the shapes in chapter 3 transfer well, the specifics do not.

This is not financial advice, and it is not a recommendation to buy any commitment. Chapter 11 explains how reserved and committed pricing works structurally, and deliberately stops short of telling you to buy one, because that decision depends on forecasts only you have.

One honest limitation. Because this book avoids specifics, it cannot tell you whether a particular line on your bill is high. It can tell you what that line is, what drives it, and how to compare it against your own previous months — which is the comparison that actually matters, and the only one that survives a price change.

CHAPTER 3

The five things you actually pay for

Almost every AWS charge is one of five shapes. Learn the shapes and an unfamiliar service becomes readable from its pricing page in a couple of minutes.

The five shapes

Time. You pay for how long a resource exists, usually per second or per hour, whether or not it is doing anything. Compute instances, databases, load balancers, public IPv4 addresses. The quantity you control is *duration* and *size*.

Storage. You pay for how much data you keep and for how long, usually per gigabyte-month. Object storage, disks, snapshots, backups, log retention. The quantity you control is *volume* and *retention*.

Requests and operations. You pay per call, usually per million. Object reads and writes, queue messages, function invocations, API calls, database operations. The quantity you control is *how chatty your system is* — and this is the shape most often multiplied by a loop nobody costed.

Data transfer. You pay for bytes moving, and crucially the price depends entirely on *which boundary they cross*. Chapter 7 is devoted to it because it is the shape that behaves least like intuition.

Provisioned capacity. You pay for capacity you reserved whether or not you used it: provisioned throughput, pre-warmed concurrency, allocated IOPS, committed capacity. The quantity you control is *the reservation*, and the risk is that it silently decouples from reality.

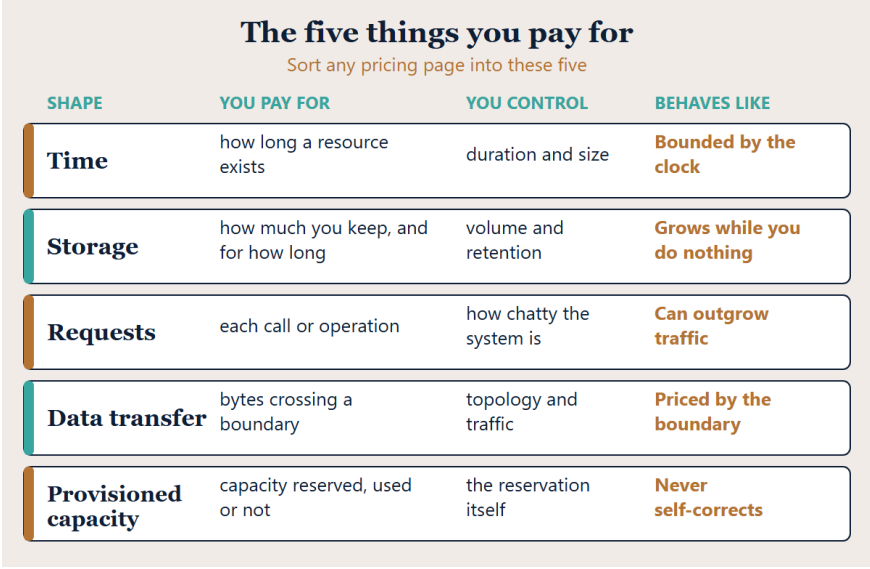


Figure 1. The five pricing shapes, and how each behaves as you grow.

Reading an unfamiliar service

Take any AWS service you have not used. Open its pricing page and sort what you see into the five shapes. You will usually find two or three, and one of them will dominate for your workload.

A managed queue: requests dominate. A data lake: storage dominates, with requests second and a transfer surprise if consumers sit elsewhere. A managed database: time dominates, with storage second and provisioned IOPS a third if someone set it. A serverless function: requests and time-multiplied-by-memory, with transfer appearing the moment it talks to anything outside.

That exercise is the whole skill. Everything else is detail.

Try it on services you will meet first. An EC2 instance is billed for time (by the second [S23]), plus the EBS volumes attached to it, which are charged by the gigabytes you provision [S31], plus any data it sends out. An S3 bucket is storage, requests and retrievals, plus transfer if the data leaves [S16]. A Lambda function is requests plus duration scaled by memory [S22]. CloudWatch Logs is charged for collecting and ingesting logs, storing them and analysing them [S21]. Each has its own pricing page, and the sorting exercise is the same every time.

Why the shape matters more than the rate

Because the shapes behave differently as you grow.

Time scales with how long things run, so it is bounded by the clock and is the easiest to predict. Storage **accumulates** — it is the only shape that grows while you do nothing, which is why retention is a cost decision and not a housekeeping one. Requests scale with traffic *and* with architectural chattiness, so they can grow superlinearly when a change adds calls per unit of work. Transfer scales with traffic and with topology. Provisioned capacity does not scale with anything — it sits where you left it, which is its entire danger.

A workload whose cost is dominated by storage and one dominated by requests need completely different attention, and knowing which you are is worth more than any individual saving.

The one that catches everyone

Requests multiplied by a loop. Per-call pricing is so small per call that it reads as free, so nobody models it. Then a retry policy, a polling interval, a per-row call inside a per-row loop, or a health check that runs on every instance every second turns a rounding error into a line item.

The database world has a name for this pattern — the N+1 query — and the billing version is identical in shape and much more expensive to discover, because the feedback is a month away rather than a slow page.

When it goes wrong

"A service appears on the bill that we never provisioned." Many services are consumed implicitly: logging, metrics, key management, DNS queries, data processed by a load balancer. You provisioned the thing that calls them.

"Our cost went up but usage is flat." Check retention and accumulation. Storage grows at the rate you write, and bills on what has accumulated, so a flat write rate still produces a rising bill until something expires it.

"We cannot find where the requests come from." Look for the shortest interval in the system: health checks, polling loops, retries with no backoff, and any per-item call inside an iteration.

"The pricing page lists units we do not recognise." Normalise to the five shapes before trying to estimate anything. If a unit does not fit, it is usually provisioned capacity wearing a service specific name.

Checklist

- You can sort any pricing page into the five shapes.
- You know which shape dominates each of your significant workloads.
- You treat storage as accumulating and therefore retention as a cost control.
- You have looked for per-call charges sitting inside a loop.
- You recognise provisioned capacity as the shape that does not self-correct.